

OneNote API Fails with Large SharePoint Document Library – Despite File Type and HTML File Type Being True



Jan Winter
5

May 18, 2025, 9:40 AM

We are encountering an issue with the Microsoft Graph API when trying to access OneNote content. Specifically, when the associated SharePoint Document Library contains a large number of items, our API calls fail—even though the OneNote notebook itself only has a small number of pages (e.g., 3 pages totaling ~20 items).

We found the following Microsoft blog post describing this issue: <https://learn.microsoft.com/de-de/archive/blogs/onenotedev/onenote-api-calls-fail-with-a-large-number-of-items-in-a-sharepoint-document-library>

Based on this guidance, we ran the recommended diagnostic tool and received this output:

YAML

```
List: Documents
Items Count: 117190
Notebooks Count: 1
Folders Count: 1
Sections Count: 2
Content Type ID: True
File Type: True
HTML File Type: True
```

According to the article, the item count being over 20,000 should only be a problem if either File Type OR HTML File Type is False—which in our case, both are True.

Request:

Why are our OneNote API requests still failing under these circumstances?

Are there additional thresholds or undocumented limitations we should be aware of?

Is there a workaround or best practice for accessing OneNote content in document libraries

with high item counts, even if the actual notebook is small?

We would appreciate any insights or recommendations on how to proceed.

Thank you.

Microsoft Security | Microsoft Graph

 0 comments

 [Report a concern](#)

 [I have the same question \(1\)](#)

[Sign in to comment](#)

1 answer

Sort by: [Most helpful](#) 



Anonymous

May 29, 2025, 5:49 PM

Hi [Jan Winter](#),

Thanks for reaching out to Microsoft!

You've correctly noted that the documented threshold of 20,000 items in a SharePoint Document Library should not affect OneNote API access when both `File Type` and `HTML File Type` are `True`. However, based on our investigation and internal findings, there are a few additional factors that may be contributing to the issue you're experiencing:

Undocumented Limitations: Even when the notebook itself is small, OneNote API calls can fail if the section resides in a large document library. Microsoft has acknowledged that support for large sections is not fully implemented, and this can cause unexpected failures.

Rate Limiting and SharePoint Throttling: Libraries with high item counts (like yours, with over 117,000 items) may be subject to throttling or rate limits, especially if accessed frequently or via automation. This can result in intermittent or persistent API failures.

Best Practices and Workarounds:

- Split the notebook into more sections to reduce the load per section.
- Move the notebook to a smaller document library if feasible.
- Use direct OneNote URLs from the desktop app to bypass some API dependencies.
 - Implement retry logic with exponential backoff in your API calls to handle transient failures.

Hope this helps.

If the answer is helpful, please click **Accept Answer** and kindly upvote. If you have any further questions about this answer, please click **Comment**.

Was this answer helpful?

 Yes

 No

 1 comment

 Report a concern



Jan Winter

5

Jun 1, 2025, 10:59 AM

Hi Mohan Joshi,

Thank you for your response.

I'd like to clarify a few points and share my concerns, as the current explanation remains difficult to communicate to my clients.

Notebook size: As mentioned in my initial message, the OneNote notebook in question is extremely small. It contains two sections with just one page each, created purely for testing purposes. There's nothing complex or nested — this structure is already minimal and cannot be split further.

Understanding throttling: To my knowledge, throttling and rate limits are generally tied to the number of API requests and the payload size of the responses, not the total number of items in the underlying SharePoint document library. If throttling were based solely on item count, this would significantly hinder the scalability of solutions built on SharePoint. While I can easily explain to clients that sending "1,000 requests per minute is too much," it's far harder to tell them: "You cannot access a notebook because the library contains over 100,000

items — even if your notebook has only three pages."

Client and developer experience: I regularly work with clients to help them integrate Microsoft Graph APIs and Microsoft 365 services. When undocumented or unexplained API limitations arise — especially those tied to internal architecture — they not only hinder the reliability of our solutions, but also erode trust in Microsoft's platform. I'm not just looking for a workaround — I'm trying to understand why this behavior occurs so that I can provide clear and confident explanations to our clients and continue advocating for Microsoft-based solutions.

**** My key question:**** Could you please help me understand the technical relationship between the size of a SharePoint document library and OneNote API failures — especially when accessing a notebook that is small, static, and contains minimal content? Is there any internal documentation or architectural explanation that can shed light on this behavior?

This would allow me to better advise my clients, rather than falling back on vague statements or shifting blame, which helps no one — not me, not Microsoft, and certainly not our shared customers.

Thank you again for your support.

Best regards, Jan Winter

 0 votes 

[Sign in to comment](#)

[Sign in to answer](#)